Solution: GPU/SIMT Programming

CUDA Indexing, SIMT Divergence, and Bandwidth

1. The launch creates:

   ```
   16 blocks * 64 threads/block = 1024 threads
   ```

   The valid indices are `0` through `999`, so 1000 threads actually write to `out`. The remaining 24 threads compute out-of-range indices and do not write.

2. The thread computes:

   ```
   idx = blockDim.x * blockIdx.x + threadIdx.x
       = 64 * 5 + 17
       = 337
   ```

3. The last block has `blockIdx.x = 15`, so its threads compute:

   ```
   idx = 64 * 15 + threadIdx.x = 960 + threadIdx.x
   ```

   The threads with `threadIdx.x = 40` through `63` compute `idx = 1000` through `1023`. These threads fail the `idx < n` guard and do no useful work.

4. Yes. The first warp of block 0 covers `idx = 0..31`. The 16 even-indexed lanes take the `out[idx] = 2.0f * v` branch, and the 16 odd-indexed lanes take the `out[idx] = v + 1.0f` branch.

   Under SIMT execution, the warp must execute both branch paths. During each path, only the lanes belonging to that path are active, while the other lanes are disabled. Thus each branch executes with 16 active lanes.

5. The second warp of the last block covers `idx = 992..1023`. Only `idx = 992..999` pass the outer `idx < n` guard, so only 8 lanes are active at all; the other 24 lanes are disabled.

   Among those 8 valid lanes, 4 have even indices and 4 have odd indices. Therefore, the inner branch executes the even path with 4 active lanes and the odd path with 4 active lanes.

6. One equivalent single-threaded CPU version is:

   ```
   void vecTransform_cpu(const float *x, const float *y, float *out, size_t n) {
       for (size_t idx = 0; idx < n; idx++) {
           float v = x[idx] + y[idx];

           if (idx % 2 == 0)
               out[idx] = 2.0f * v;
           else
               out[idx] = v + 1.0f;
       }
   }
   ```

   In the CUDA version, many GPU threads are launched, and each thread computes one `idx`. In the traditional CPU version, one CPU thread explicitly loops over all valid `idx` values from `0` to `n - 1`.

7. Each valid element performs:
   - 2 global memory reads: `x[idx]` and `y[idx]`
   - 1 global memory write: `out[idx]`

- 2 floating-point operations: one addition for `v`, then either one multiplication or one addition

Each `float` is 4 bytes, so each valid element moves approximately:

```
3 * 4 bytes = 12 bytes
```

The operational intensity is:

```
2 FLOPs / 12 bytes = 1/6 FLOPs/byte ~= 0.167 FLOPs/byte
```

This kernel is more likely to be bandwidth-bound than compute-bound because it performs very little arithmetic for each byte moved from global memory.